HOTSPA 2026

CGLA上の多様なAIワークロードに向けた統合ソフトウェアスタックの実装と今後の展望

Implementation and Future Directions of an Integrated Software Stack for Diverse AI Workloads on CGLA

安藤 拓翔　　宗片 吉史　　中島 康彦

奈良先端科学技術大学院大学

背景: AIワークロードの多様化とソフトウェアスタックの必要性

AIワークロードの多様化

LLM推論，音声認識(ASR)，画像生成(Diffusion)など

アクセラレータには単一用途ではなく汎用性が必要

現状の課題

アプリごとにAPI・データ配置・実行制御・評価方法が分断

ソフトウェア資産の再利用が困難

本稿の目的

(1) 異種アプリに共通する律速要因の特定

(2) 再利用を妨げている箇所の明確化

(3) 共通化すべき層と方策化すべき層の整理

本稿の貢献

貢献1: LLM・ASR・Diffusion実装を横断整理し，
CGLAソフトウェアスタックに共通する論点を抽出

貢献2: エッジ/サーバをまたぐ将来基盤を，
共通インタフェース層と方策別実行層に分けて記述する枠組みを提示

貢献3: 次段階の研究課題を具体化
Python連携機構，計測基盤，3D積層DRAM世代への接続

位置づけ: 新しいカーネルや単独アプリの性能報告ではなく，
「何を共通化し，何を方策として切り分けるべきか」を整理する設計論文

IMAX/CGLA の概要

CGLA = 粗粒度リニアアレイASIC

PEとLMMを一次元に交互配置

隣接PE間のパイプライン実行

ダブルバッファによる演算/DMA転送オーバラップ

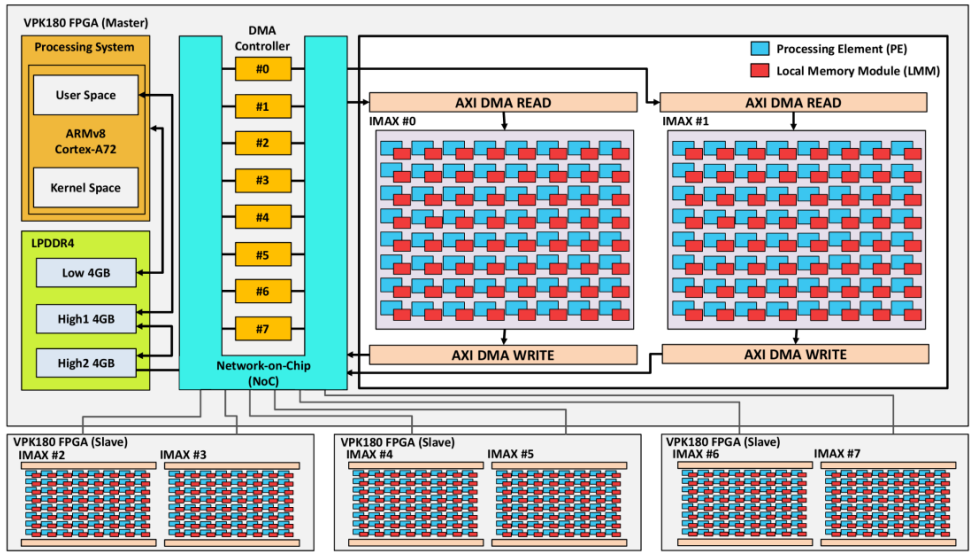
IMAX3 実機構成

PS: Arm Cortex-A72 (ホスト)

PL: 8-lane CGLA本体

DMAでデータをLMMへ搬送→カーネル実行

ソフトウェアスタックの対象: PS/PL境界をまたぐ設計



既存成果の整理: 3つのAIワークロード

題材	出典	主要結果	スタックへの含意
Stable Diffusion	MCSoC	F32: 30.7%, F16: 59.0%, Q3_K: 10.3% → 単一データ型前提では不可	データ型混在を吸収するAPI バックエンド選択機構が必要
Llama3 Q6_K	SOCC	E2E latency 34.2s→30.4s (約11%短縮) Q6_K新規offload	offload境界設計を runtimeに組み込む必要
Whisper ASR	CANDAR	Jetson比 1.90× 省エネ RTX4090比 9.83× 省エネ EXEC比率: FP16で60.89%	位相別テレメトリと LMM/burst調整の一体化

共通課題: データ移動の支配性

LLM推論の発見

prefill/decode共にDMA・バッファ配置・chunk境界・copyoutが支配的

演算器の理論性能よりも**データ移動**が律速

decodeはLOADとホスト側処理に支配される

Q-Snapの成果

量子化認識動的チャンキングによりprefill 1.62× 高速化

ランタイム層が性能改善の主戦場であることを実証

ソフトウェアスタック設計への影響

① prefill/decode分離観測テレメトリが不可欠

② 量子化形式・LMM容量考慮のchunkサイズ切替ランタイム方策

→ 中核 = テレメトリ + chunk方策 + ホスト協調制御

共通課題: ASR・Diffusion実装からの知見

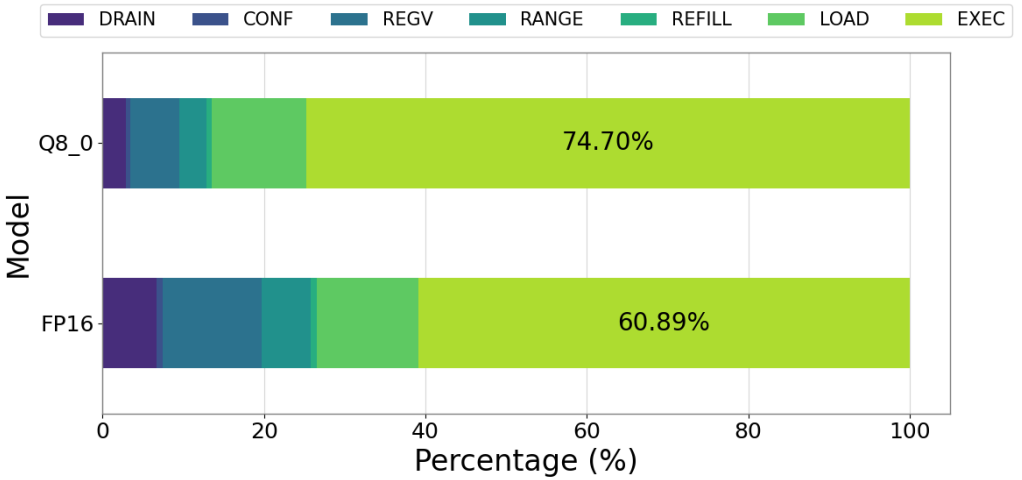
Whisper ASR (CANDAR)

FP16 dot積を新規実装

FP16→FP32変換+SIMD+マルチスレッド

ホスト側処理が多く，完全オフロードではなく混合実行が自然

Jetson比 **1.90×** / RTX4090比 **9.83×** 省エネ



Stable Diffusion (MCSoC)

量子化dot積カーネルをLLMから画像生成へ再利用

データ型混在:
F32(30.7%)+F16(59.0%)+Q3_K(10.3%)

量子化中心の議論だけでは不十分

共通課題: Python OSSとC/C++実装の断絶

観点	Python OSS側の自然な表現	現行CGLA実装での個別対応
演算呼び出し	演算グラフ，モジュール呼び出し 推論ループ	dot積・行列演算への 手作業マッピング
メモリ管理	Tensor objectと 自動確保/解放	LMM容量を意識した分割と DMA制御
計測取得	profiler hook, ログcallback	位相別時間計測, copyout回数，chunk記録
実行切替	Python側での backend指定	ホストCPU実行と CGLA実行の分岐実装

→ 共通API・共通ログ形式がないと各アプリが独立実装として閉じてしまう

提案: CGLAソフトウェアスタックの5層構造

層	主な役割	IMAX3実機での対応先
アプリケーション層	モデル実行，前後処理 オフロード点の記述	主要OSS ホスト側制御コード
フロントエンド /API/IR層	演算単位，テンソル境界 呼び出し規約の統一	C/C++ラッパ カーネル呼び出しAPI
ランタイム方策 /テレメトリ層	DMA制御，分割実行 計測，ログ収集	ホストCPUスケジューラ DMA起動部，時間計測
CGLAバックエンド層	CGLA向けカーネル実行 データ配置	IMAX3 lane, PE, LMM 実行ライブラリ
IMAXハードウェア層	物理資源	VPK180 SoC PS(ARM)+PL(CGLA)

設計方針: 共通フロントエンド＋方策特化バックエンド

共通化の3つの対象

① フロントエンドAPI / IR

テンソル入出力境界，バッファ量，オフロード候補，テレメトリ点

② プロファイリング・テレメトリの共通形式

DRAIN, CONF, REGV, LOAD, EXEC の位相分離

③ ランタイム方策の制御点

chunk, バッファ, DMAの制御

方策特化バックエンド

エッジ: 省メモリ・低電力重視（Whisper ASR等）

サーバ: 高帯域活用・高スループット重視（長文脈LLM等）

→ 同一アプリを共通記述し，構成ごとに異なる方策を適用

共通フロントエンドと方策特化バックエンドの責務分担

層	共通化する責務	構成ごとに変える責務
フロントエンド	テンソル境界，オフロード候補演算 プロファイリング点，入出力契約	アプリ固有ラッパの 薄い差分
ランタイム	位相計測形式，chunk/transfer の記録形式，ログ収集API	chunk方策，バッチ化 先読み，copyout方策
バックエンド	カーネル呼び出し規約 バッファ管理の基本契約	データ型別カーネル選択 エッジ/サーバ向け資源配分

ポイント:

完全共通化は非現実的 — エッジとサーバでメモリ・電力・帯域制約が異なる

共通記述形式＋方策特化により，アプリ追加コストを大幅に削減

テレメトリ駆動ランタイムの具体例

テレメトリ = 一級の設計要素

imax-nanosec-stats による位相分離表示

DRAIN, CONF, REGV, LOAD, EXEC を個別計測

ログ出力 = 補助機能ではなく設計のコア

DMA改善の実例 (Qwen3 0.6B Q8)

1 chunkごとにCPYOUT → まとめ発行へ変更

DRAIN実効速度: **0.264 → 1.077 Gbps**

E2E実行時間: **97.2s → 87.2s**

→ DMA制御やcopyout設計がスタック上の独立した研究課題

必要な観測項目

DMA/load/copyout 時間

chunkごとの待ち時間

prefill/decode分離ログ

バッファ使用量・転送量

→ 帯域考慮ランタイム

→ KVキャッシュ配置研究

の基盤にもなる

エッジ構成 vs サーバ構成

エッジ構成

優先事項:

低消費電力実行

限られたメモリでの安定動作

バックエンド方策:

小さなLMM容量に合わせたchunk設定

ホスト/アクセラレータ混合実行

低遅延DRAIN制御

対応アプリ: Whisper ASR等

サーバ構成

課題:

帯域増加だけでは性能を引き出せない

ホスト側オーバーヘッドが新たな律速に

バックエンド方策:

メモリ階層を意識したランタイム

長文脈LLM向けKVキャッシュ管理

複数要求サービング

将来: 3D積層DRAM世代への接続

現行の実行フローと共通化の方向性

現行の実行フロー（全ワークロード共通）

Step 1: アプリ層で推論・前後処理 → dot積・行列演算をオフロード候補として切り出し

Step 2: C/C++ラッパがテンソル形状・データ型・LMM制約からchunkサイズ/DMA計画を決定

Step 3: ランタイムがLOAD→設定転送→実行→copyoutを順に起動

Step 4: テレメトリがDRAIN/CONF/LOAD/EXEC時間を収集

課題: フロー自体は存在するが表現形式がアプリごとに異なる

必要なのは新しい層の追加ではなく，

既存の呼び出し手順・観測点・調整パラメータを

アプリ横断で共通記述に落とし直すこと

今後の展望: 3つの研究課題

課題1: Python連携機構

AI OSSの入口(Python)とCGLA実装層(C/C++)の橋渡し

目標: Python環境の移植ではなく，オフロード候補検出と実行記録抽出の標準化

課題2: ランタイム方策の定量評価

適応的chunking・帯域考慮スケジューリングの実装と固定方策との比較

token/s等の外部指標だけでなく，chunk回数・copyoutサイズ・LOAD/EXEC比率も報告

LLM・ASR・Diffusionで方策の一般性を比較

課題3: 3D積層DRAM世代への接続

サーバ側展開に直結する論点

単なるカーネル最適化ではなく，メモリ階層を意識した基盤設計

将来メモリ階層を取り込める抽象化

HW帯域増加の効果を引き出すにはSWスタック再設計が必要

まとめ

本稿の整理

LLM・ASR・画像生成のCGLA実装を横断比較

共通課題の特定

① **メモリ移動の支配性** — 演算器性能よりもDMA/copyoutが律速
② **Python OSS / C++実装の断絶** — 再利用を阻害
③ **アプリ個別最適化** — 資産が蓄積しない

提案した層構造

共通フロントエンド＋計測駆動ランタイム＋方策特化バックエンド

今後の課題

Python連携機構 / ランタイム方策の定量評価 / 3D積層DRAM世代接続

付録 (Appendix)

質疑応答用の補足資料

付録: IMAX3/IMAX4 ハードウェア構成

IMAX3 (エッジ向けプロトタイプ)

FPGA: Xilinx VPK180

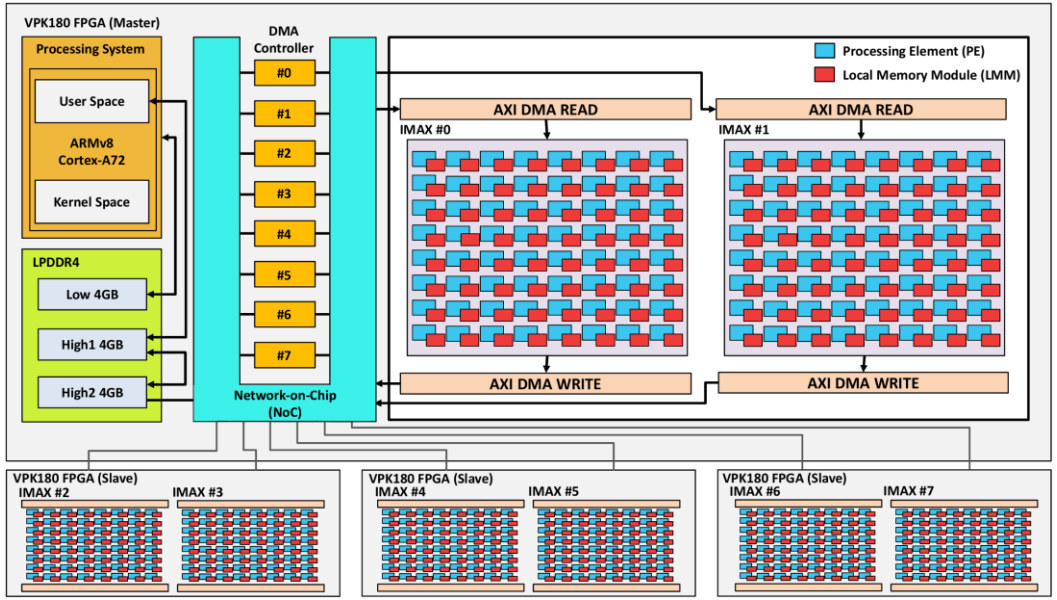
PS: Arm Cortex-A72 (1.5GHz)

PL: 8-lane CGLA

各lane: 64 PE + 64 LMM

LMM: 64KB/unit, ダブルバッファ

DMA: AXI4 経由



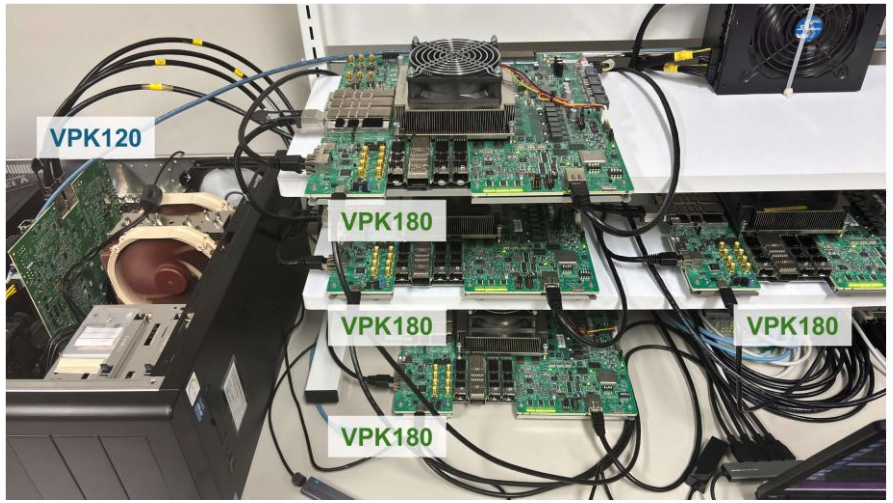
IMAX4 (サーバ向けプロトタイプ)

FPGA: Xilinx VPK120

PS: Xeon (ホストPC側)

PL: 4-lane CGLA (PCIe接続)

高帯域メモリ活用が将来課題



付録: PE構造とメモリ階層

PE (Processing Element) 構造

複数演算器 + アドレス生成器

隣接PE間でデータ受け渡し

パイプライン深度 = lane長

LMM (Local Memory Module)

PEに隣接配置

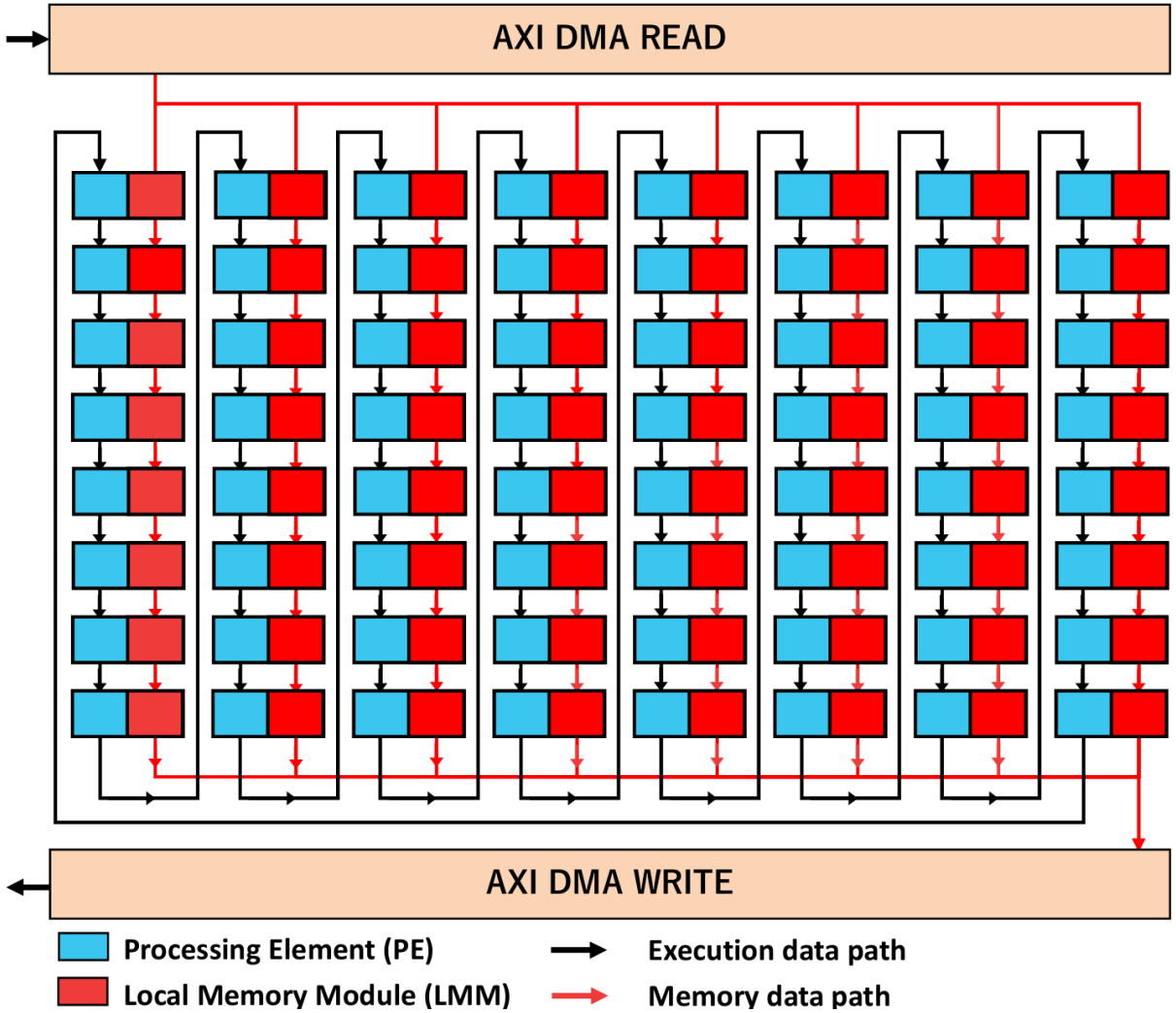
ダブルバッファ: 演算/DMA転送オーバラップ

サイズ制約がchunk設計に直結

メモリ階層

Main Memory → DMA → LMM → PE

転送律速がSWスタック設計の核心



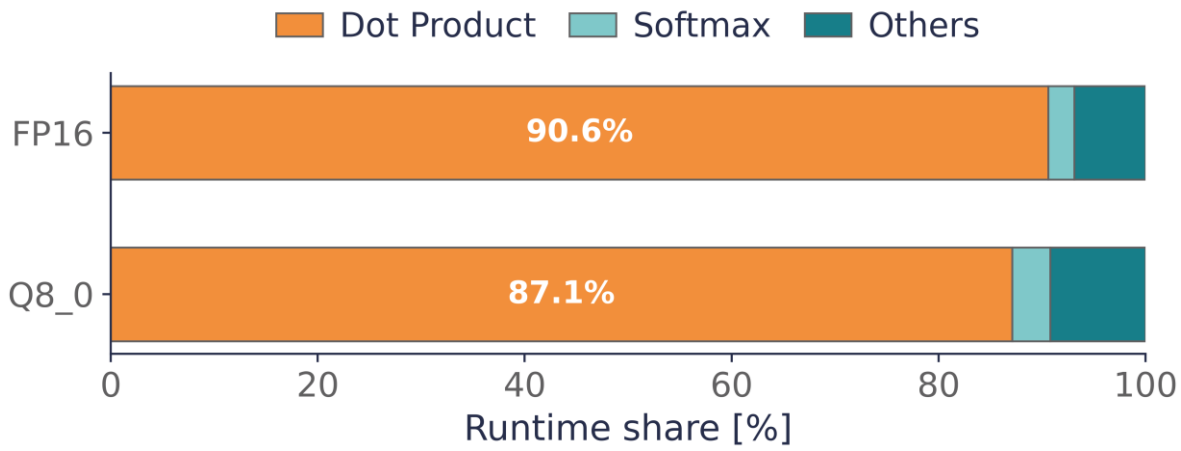
付録: LLM実行時間内訳 (Decode/Prefill)

Decode (トークン生成)

小粒度反復 → 転送/同期/copyoutの固定費が累積

LOADとホスト側処理が支配的

演算器utilization だけでは改善余地を捉えきれない



Prefill (初期処理)

Q-Snap: 量子化認識動的チャンキング

1.62× 高速化を達成

chunk方策がランタイム層の研究課題であることを実証

DRAIN改善 (Qwen3 0.6B Q8)

毎chunk CPYOUT → まとめ発行

DRAIN: 0.264 → 1.077 Gbps (4.08×)

E2E: 97.2s → 87.2s (10.3%短縮)

付録: Whisper ASR 詳細結果

実装内容

whisper.cpp上でFP16 dot積を新規実装

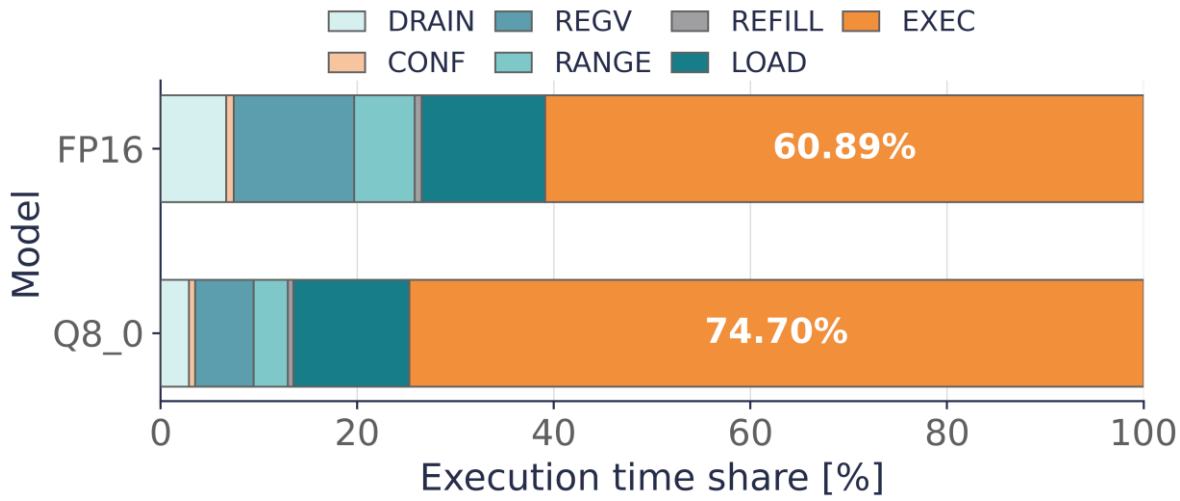
FP16→FP32変換 + SIMD + column-wise multithreading

EXEC比率 (IMAX実行時間内訳)

FP16: 60.89%

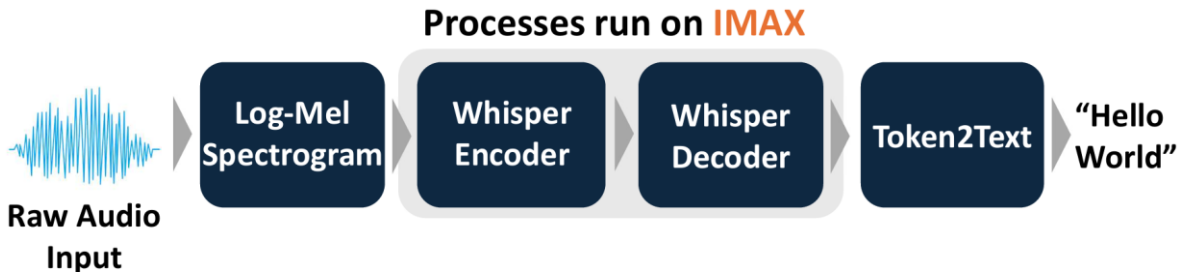
Q8_0: 74.70%

→ EXEC比率が高い = CGLA活用が効果的

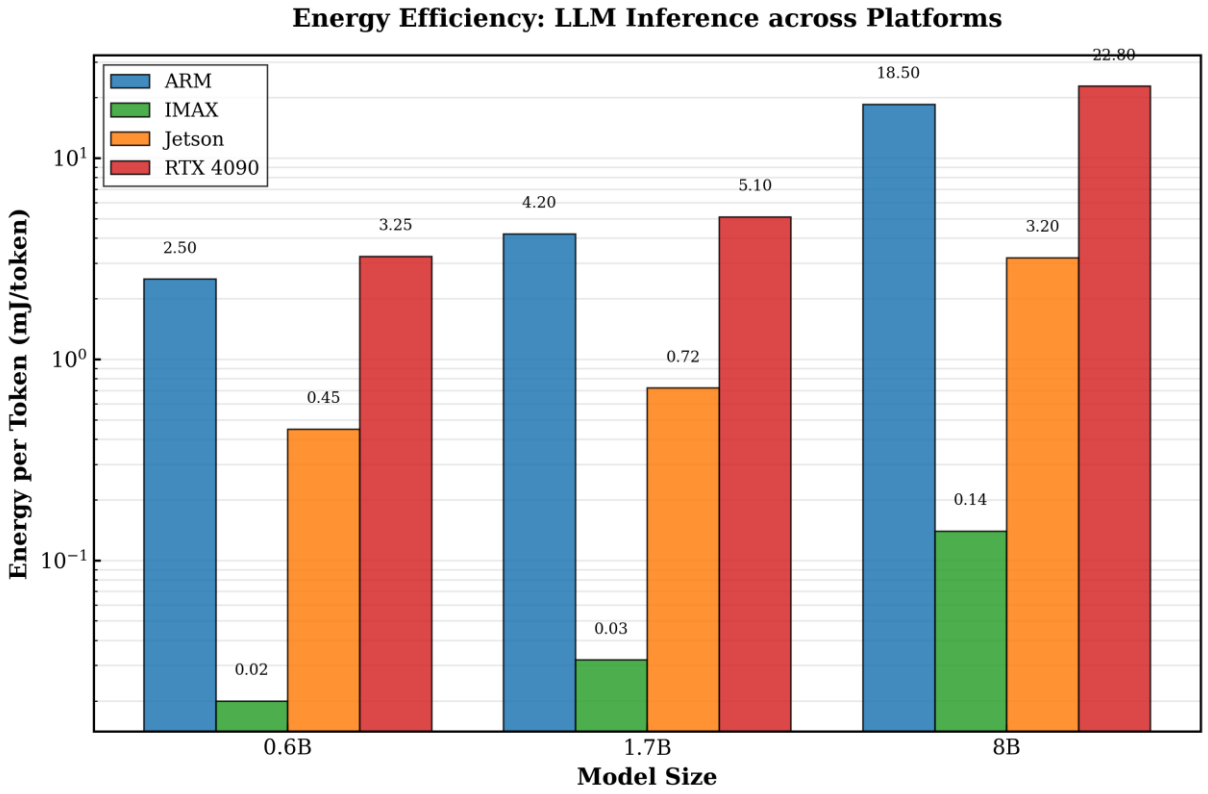


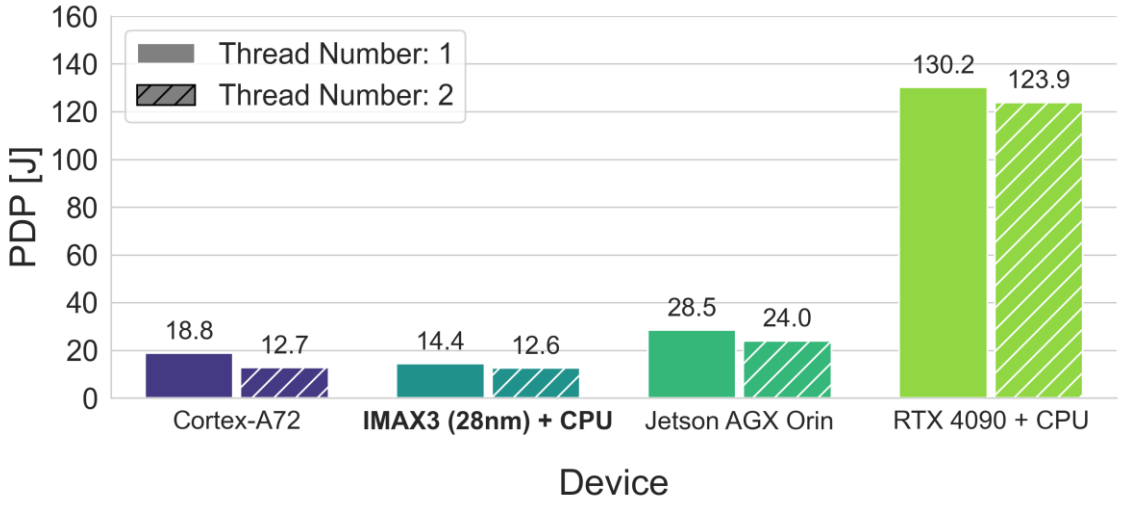
省エネルギー性能 (whisper-tiny.en)

比較対象	省エネ倍率
vs Jetson AGX Orin	1.90×
vs RTX 4090	9.83×



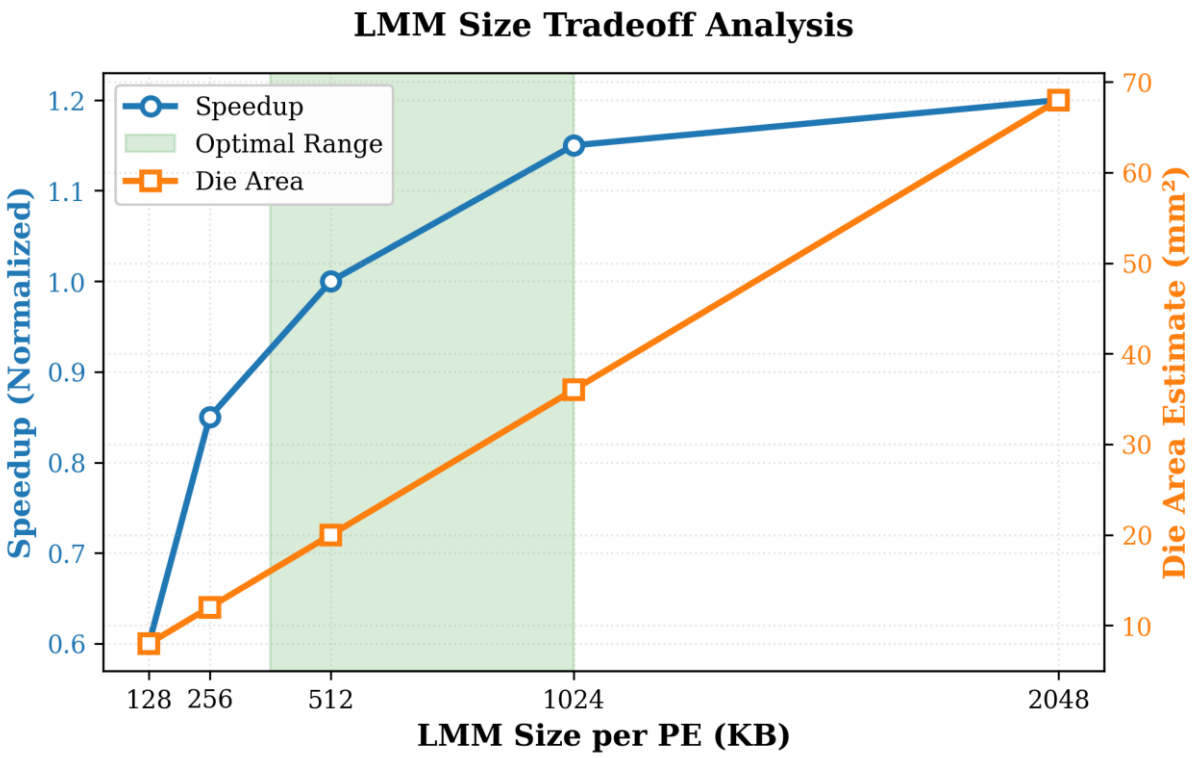
付録: プラットフォーム間比較

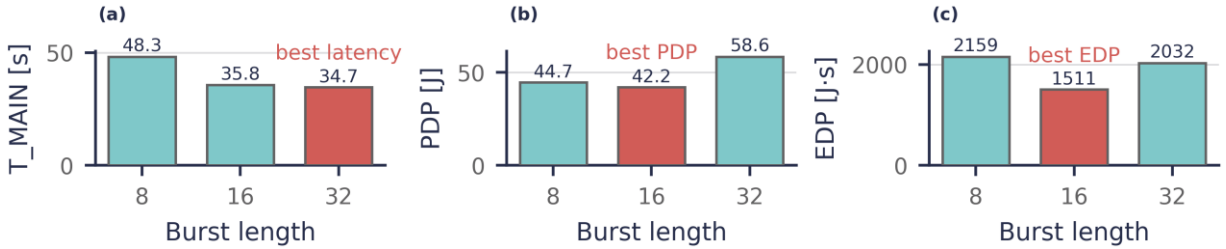




CGLAの強み: ASIC級の電力効率でGPUより桁違いに低消費電力，かつ多様なワークロードに対応

付録: LMMサイズ・burst設定のトレードオフ

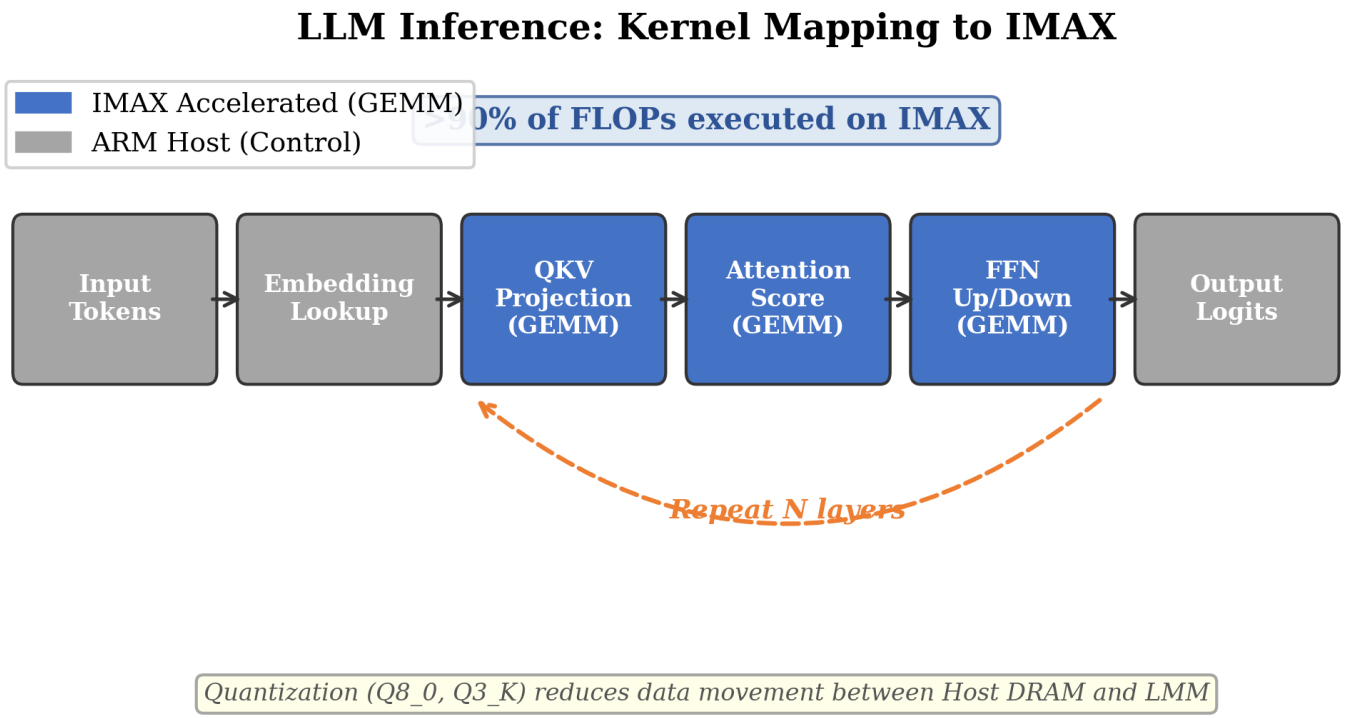




LMMサイズとburst設定はランタイム方策の主要パラメータ

chunk粒度・転送効率・メモリ使用量のバランスをSWスタックが制御

付録: LLMカーネルマッピングフロー



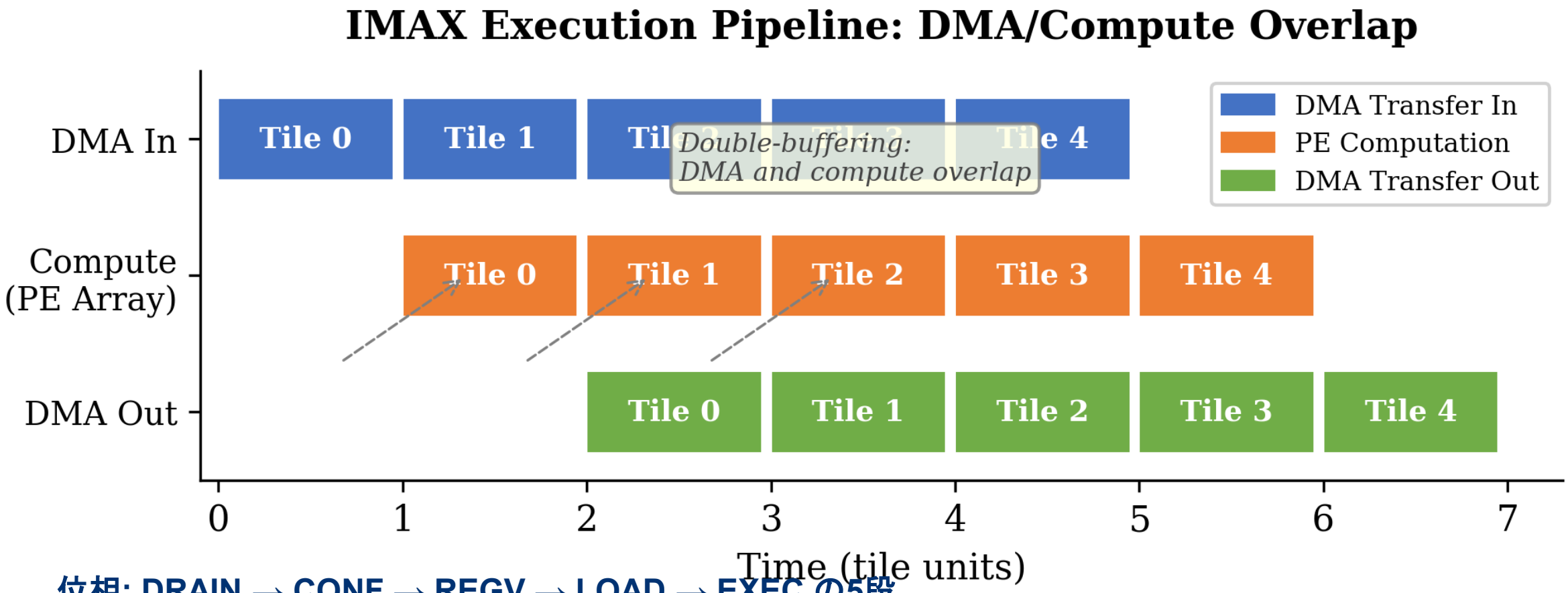
カーネルマッピング

量子化形式ごとに
専用カーネルを実装

offload対象:
Q8_0, Q3_K, Q6_K
FP16 dot積

SWスタックの役割:
どの演算を検出して
オフロードするか
= 実装境界の設計

付録: IMAX実行パイプラインタイムライン

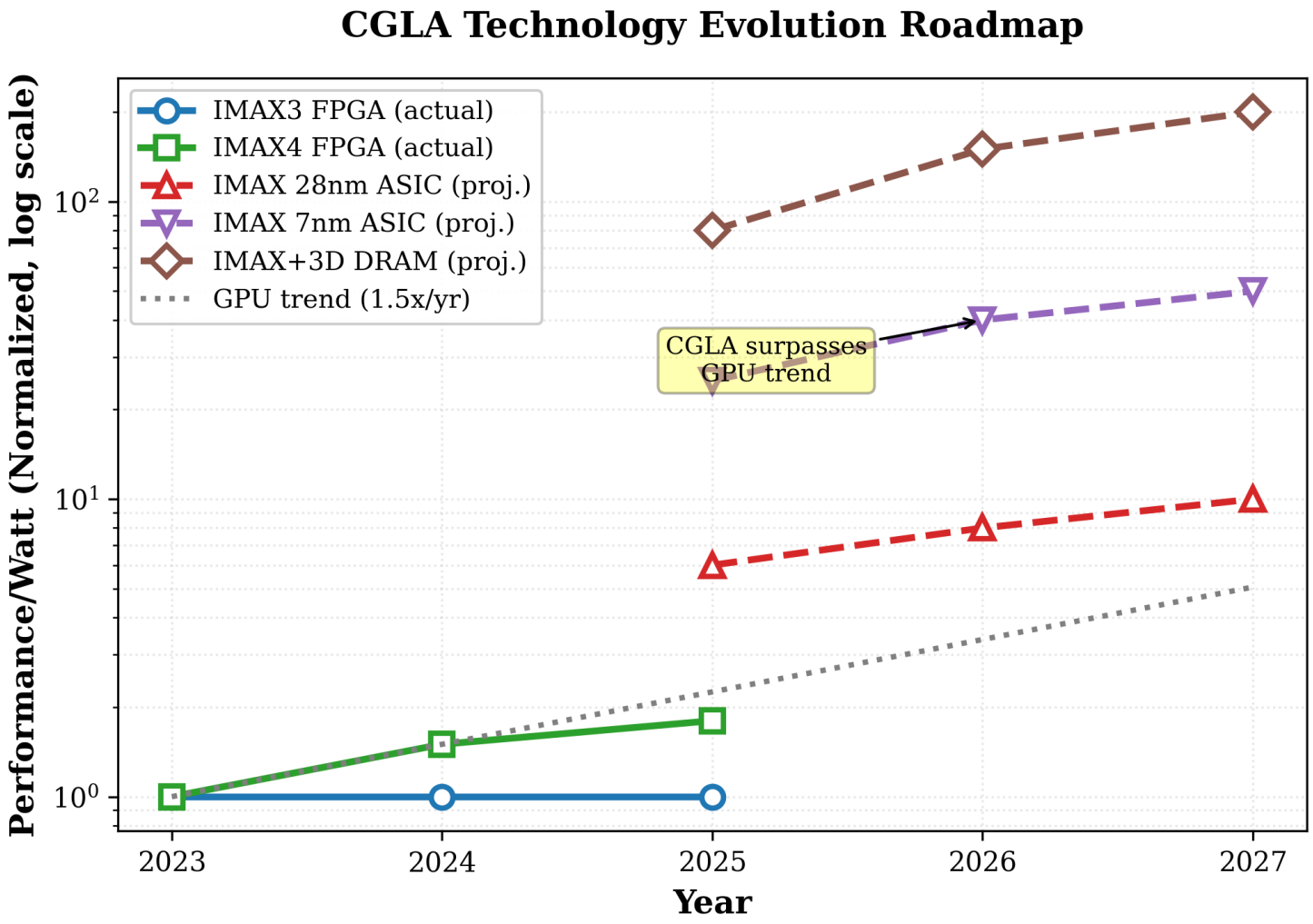


位相: DRAIN → CONF → REGV → LOAD → EXEC の5段

ダブルバッファによる演算/転送オーバラップが理想だが,

実際にはchunkサイズ・LMM容量制約により待ちが発生 → テレメトリで可視化

付録: 3D積層DRAM世代へのロードマップ



サーバ側CGLA展望

3D積層DRAM搭載により
大幅な帯域向上が期待

しかし:
HW帯域増加 ≠ 性能向上
ホスト側ランタイムが
新たな固定ボトルネックに

→ SWスタック再設計が
HW拡張の効果を
引き出す鍵

付録: 関連業績一覧

本稿で参照した既発表論文

[IEEE Access 2025] T. Akabe et al., "IMAX: A Power-Efficient Multilevel Pipelined CGLA and Applications"

→ **IMAX アーキテクチャの基礎**

[MCSoC 2025] T. Ando et al., "Implementation and Evaluation of Stable Diffusion on a General-Purpose CGLA Accelerator"

→ **Stable Diffusion実装 — データ型混在の発見**

[CANDAR 2025] T. Ando et al., "Energy-Efficient Hardware Acceleration of Whisper ASR on a CGLA"

→ **Whisper ASR — Jetson比1.90×, RTX4090比9.83× 省エネ**

[IEEE Access 2025] T. Ando et al., "Efficient Kernel Mapping and Comprehensive System Evaluation of LLM Acceleration on a CGLA"

→ **LLM系統評価 — decode律速の発見**

[SASIMI 2025] T. Ando et al., "A Detailed Analysis of LLM Execution on IMAX3 and Initial Evaluation of IMAX4"

→ **LLM詳細分析 — ARMホスト側オーバーヘッド**

[ICISN 2026] T. Ando et al., "Q-Snap: Quantization-Aware Dynamic Chunking for LLM Execution on a CGLA"

→ **Q-Snap — prefill 1.62× 高速化**